

PROJECT REPORT

SYSTOLIC ARRAY FOR LOW-COST FPGA-BASED CNN ACCELERATION

Team members: _____

Supervisor: _____

Class / Course: _____

Academic year: 2025 - 2026

TABLE OF CONTENTS

1. INTRODUCTION

- 1.1. Project Objectives
- 1.2. Scope of Work
- 1.3. Team Roles and Responsibilities

2. THEORETICAL BACKGROUND

- 2.1. Systolic Array Architecture
- 2.2. Processing Element (PE) Design Improvements
- 2.3. Frequency and Layout Challenges
- 2.4. Estimation and Design Space Exploration (DSE) Tools
- 2.5. Flexibility and Scalability

3. SYSTEM DESIGN

- 3.1. General Block Diagram
- 3.2. Processing Element Module (pe_mac.v)
- 3.3. Systolic Array Module (systolic_array.v)
- 3.4. Verification Testbench (tb_systolic_array.v)

4. SIMULATION RESULTS

- 4.1. Comparison Results
- 4.2. Remarks

5. CONCLUSION AND FUTURE WORK

- 5.1. Conclusion
- 5.2. Future Work

REFERENCES

1. INTRODUCTION

In recent years, Convolutional Neural Networks (CNNs) have become the core foundation of many artificial intelligence applications such as computer vision, speech recognition, and natural language processing. However, the massive computational workload of CNNs - mainly matrix multiplications and convolutions - requires dedicated hardware to achieve high performance at a reasonable energy cost.

FPGA (Field-Programmable Gate Array) is an attractive option for accelerating CNNs thanks to its reconfigurability, low latency, and better energy efficiency compared to GPUs in many edge scenarios. Among hardware acceleration architectures, the Systolic Array stands out for its regular grid structure, high data reuse, and high degree of parallelism - this is also the core architecture used in Google's TPU.

This project focuses on studying, designing, and simulating a basic Systolic Array architecture, aiming toward deployment on low-cost FPGA devices to accelerate CNN computations.

1.1. Project Objectives

- Study the operating principles of the Systolic Array architecture.
- Survey optimization techniques for the Processing Element (PE), operating frequency, and FPGA layout.
- Design and simulate a basic Systolic Array using the Verilog hardware description language.
- Verify the correctness of the design through simulation testbenches.

1.2. Scope of Work

Within the scope of this project, the team focuses on building an $N \times N$ Systolic Array at the RTL (Register Transfer Level), implementing matrix multiplication using the output-stationary model, and verifying it through functional simulation. Synthesis and actual deployment on an FPGA board (e.g., DE10-Standard / DE10-Nano) will be carried out in the next phase of the project.

1.3. Team Roles and Responsibilities

The team consists of 5 members, each responsible for a distinct area of expertise within the Systolic Array system, ensuring full coverage from overall architecture to physical implementation and real-world application:

Member	Role	Focus Area
Quan	System Architect	TPU architecture, dataflow, and instruction set
Khanh	PE Designer (Bit-level Math Engineer)	Designing the Processing Element (PE) using bit-level mathematics to reduce resources, optimal DSP48E2 configuration
Tri	Physical / Backend Engineer	Floorplanning, XDC constraint scripts, frequency optimization through the datapath
Nguyen	DSE / Tool Engineer	AFHRE and Systimator methods for resource prediction without waiting for lengthy HLS synthesis
Hung	Application Engineer (Dataflow Engineer)	Converting CNN/Transformer models into matrix multiplication (im2col), buffer management

Detailed research contribution of each member:

1.3.1. Quan - System Architect

Research findings: _____

Results / contribution: _____

1.3.2. Khanh - PE Designer (Bit-level Math Engineer)

Research findings: _____

Results / contribution: _____

1.3.3. Tri - Physical / Backend Engineer

Research findings: _____

Results / contribution: _____

1.3.4. Nguyen - DSE / Tool Engineer

Research findings: _____

Results / contribution: _____

1.3.5. Hung - Application Engineer

Research findings: _____

Results / contribution: _____

2. THEORETICAL BACKGROUND

2.1. Systolic Array Architecture

A Systolic Array is a grid of locally-connected Processing Elements (PEs), where data flows rhythmically through the array with every clock cycle (similar to a heartbeat - hence the name "systolic"). Each PE only exchanges data with its immediate neighbors, significantly reducing the need for global memory access.

Key advantages of this architecture include:

- Extremely high data reuse, reducing the required memory bandwidth.
- High throughput thanks to a high degree of parallelism.

- A regular 2D grid structure that is easy to scale and map onto hardware.

Thanks to these advantages, the Systolic Array is the core architecture of Google's TPU, achieving 15 to 30 times higher performance and 30 to 80 times better energy efficiency compared to CPUs/GPUs of the same era.

2.2. Processing Element (PE) Design Improvements

Recent research has focused on optimizing each PE internally to save FPGA resources, including the following notable techniques:

Technique	Description	Effect
Bit-level MAC	Uses AND gates, counters, and shifters instead of a traditional multiplier	Reduces energy ~99x, reduces DSP usage, increases PE throughput ~15x
LUT-based architecture	Uses lookup tables (LUTs) to skip intermediate computation steps	Significantly reduces latency
Dynamic DSP configuration	Integrates the ReLU activation function directly into the DSP slice	Saves LUT resources (used in Transformer accelerators)

2.3. Frequency and Layout Challenges

A major challenge in deploying a Systolic Array on FPGA is that CAD (Computer-Aided Design) tools often distort the regular array structure during synthesis and place & route, resulting in lower-than-expected operating frequency. Several mitigation approaches include:

- Front-end approach (segmentation): Splitting overly long DSP accumulation chains to shorten the combinational data path.
- Back-end approach (floorplanning): Manual placement combined with XDC constraint files to fix PE positions, increasing frequency by 1.29x, reaching 588 MHz on modern chip families.
- Direct DSP instantiation: Directly instantiating DSP macros instead of relying on automatic synthesis, enabling precise datapath control.

2.4. Estimation and Design Space Exploration (DSE) Tools

- AFHRE (Accurate and Fast Hardware Resources Estimation): predicts hardware resources (DSP, BRAM, LUT, FF) 40 to 610 times faster than Vivado HLS.
- Systimator: an analysis tool dedicated to low-cost FPGAs (such as Artix 7), supporting two data reuse strategies: Feature Map reuse and Filter reuse.

2.5. Flexibility and Scalability

Modern architectures aim to run multiple models without reloading the hardware. For example, Systolic-CNN uses OpenCL to create a 1-D Systolic Array architecture that is run-time reconfigurable, allowing time-sharing to process different CNN models (AlexNet, ResNet, YOLO) without recompiling the kernel.

3. SYSTEM DESIGN

3.1. General Block Diagram

The system is designed using the output-stationary model: each PE holds a fixed output element $C[i][j]$ and accumulates its value over successive clock cycles, while data from matrix A moves horizontally (left to right) and data from matrix B moves vertically (top to bottom).

The formula for computing each output element is:

$$C[i][j] = \sum (A[i][k] \times B[k][j]) \quad \text{for } k = 0..N-1$$

3.2. Processing Element Module (pe_mac.v)

Each PE performs a MAC (Multiply-Accumulate) operation and forwards data to its neighboring PEs on every clock cycle:

```
module pe_mac #(
    parameter DATA_WIDTH = 8,
    parameter ACC_WIDTH = 32
)(
    input wire clk, rst_n, clear_acc,
    input wire signed [DATA_WIDTH-1:0] a_in, b_in,
    output reg signed [DATA_WIDTH-1:0] a_out, b_out,
    output reg signed [ACC_WIDTH-1:0] c_out
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            a_out <= 0; b_out <= 0; c_out <= 0;
        end else begin
            a_out <= a_in;
            b_out <= b_in;
            c_out <= clear_acc ? (a_in * b_in)
                               : c_out + (a_in * b_in);
        end
    end
endmodule
```

3.3. Systolic Array Module (systolic_array.v)

The systolic_array module connects N x N instances of pe_mac using a generate loop, while routing the a_wire and b_wire signals between neighboring PEs:

```
generate
    for (i = 0; i < N; i = i + 1) begin : ROW
        for (j = 0; j < N; j = j + 1) begin : COL
            pe_mac #(.DATA_WIDTH(DATA_WIDTH), .ACC_WIDTH(ACC_WIDTH)) u_pe (
                .clk(clk), .rst_n(rst_n), .clear_acc(clear_acc),
                .a_in(a_wire[i][j]), .b_in(b_wire[i][j]),
                .a_out(a_wire[i][j+1]), .b_out(b_wire[i+1][j]),
                .c_out(c_out[i][j])
            );
        end
    end
endgenerate
```

3.4. Verification Testbench (tb_systolic_array.v)

The testbench automatically generates two sample 4x4 matrices A and B, computes the expected result using an ordinary software loop, then feeds data into the Systolic Array according to the mandatory "skewed" pattern of the architecture, and finally compares the output c_out against the expected values.

Data is fed in following the rule: at cycle t, row i of matrix A receives the value A[i][t-i] (if valid), column j of matrix B receives the value B[t-j][j] (if valid), and all remaining positions are zero-padded.

4. SIMULATION RESULTS

The project carried out functional simulation using the Icarus Verilog tool with a 4x4 array. The input matrices were chosen for easy manual verification:

- $A[i][j] = i \times N + j + 1$ (values from 1 to 16)
- B is a diagonal matrix with value 2 → Expected result $C = 2 \times A$

4.1. Comparison Results

Element	Simulated Value	Expected Value	Result
C[0][0]..C[0][3]	2, 4, 6, 8	2, 4, 6, 8	Pass
C[1][0]..C[1][3]	10, 12, 14, 16	10, 12, 14, 16	Pass
C[2][0]..C[2][3]	18, 20, 22, 24	18, 20, 22, 24	Pass
C[3][0]..C[3][3]	26, 28, 30, 32	26, 28, 30, 32	Pass

The simulation results show that all 16 out of 16 elements of the resulting matrix match the expected values exactly, confirming the functional correctness of the RTL design.

4.2. Remarks

- The output-stationary architecture behaves correctly according to the theoretical model presented in section 3.1.
- Pipeline latency: result $C[i][j]$ only becomes stable after data has flowed through the number of cycles corresponding to its position (i, j) in the array - this is an inherent characteristic of the systolic architecture.
- The current design is behavioral RTL; a synthesis step in Quartus and real timing verification are still needed before deployment on the DE10 board.

5. CONCLUSION AND FUTURE WORK

5.1. Conclusion

This project presented a theoretical overview of the Systolic Array architecture, PE optimization techniques, operating frequency, and FPGA design support tools. The team successfully designed a 4x4 Systolic Array at the RTL level using Verilog, simulated it, and verified its functional correctness with 100% accurate results.

5.2. Future Work

- Synthesize the design in Quartus, evaluate resource usage (LUT, DSP, BRAM), and determine the maximum operating frequency on the DE10-Standard/DE10-Nano board.
- Apply floorplanning and placement constraints to improve the operating frequency.
- Extend the design to directly support convolution for real-world CNN models (e.g., AlexNet, ResNet).
- Integrate an automatic controller to generate the skewed input pattern instead of a hand-written testbench, moving toward a complete accelerator.

REFERENCES

- [1] Frequency Improvement of Systolic Array-Based CNNs on FPGAs.
- [2] High-Frequency Systolic Array-Based Transformer Accelerator.
- [3] Systolic_CNN: An OpenCL-defined Scalable Run-time-flexible Architecture.